TASK 2: CONSTRUCTORS

Python OOP Practical Report — 15 Marks

1. Objective

To write a Python program that uses the __init__ constructor to initialize object attributes, creates objects with different values, accesses initialized attributes, and displays object information.

2. OOP Concept Used

Constructor (__init__), self, object attributes, objects, and methods.

3. Source Code

```
class Student:
    # Constructor
    def __init__(self, name, roll_no, branch, marks):
        self.name = name
        self.roll_no = roll_no
        self.branch = branch
        self.marks = marks

    # Method to display student information
    def display(self):
        print("Name:", self.name)
        print("Roll Number:", self.roll_no)
        print("Branch:", self.branch)
        print("Marks:", self.marks)
        print("--------------------")

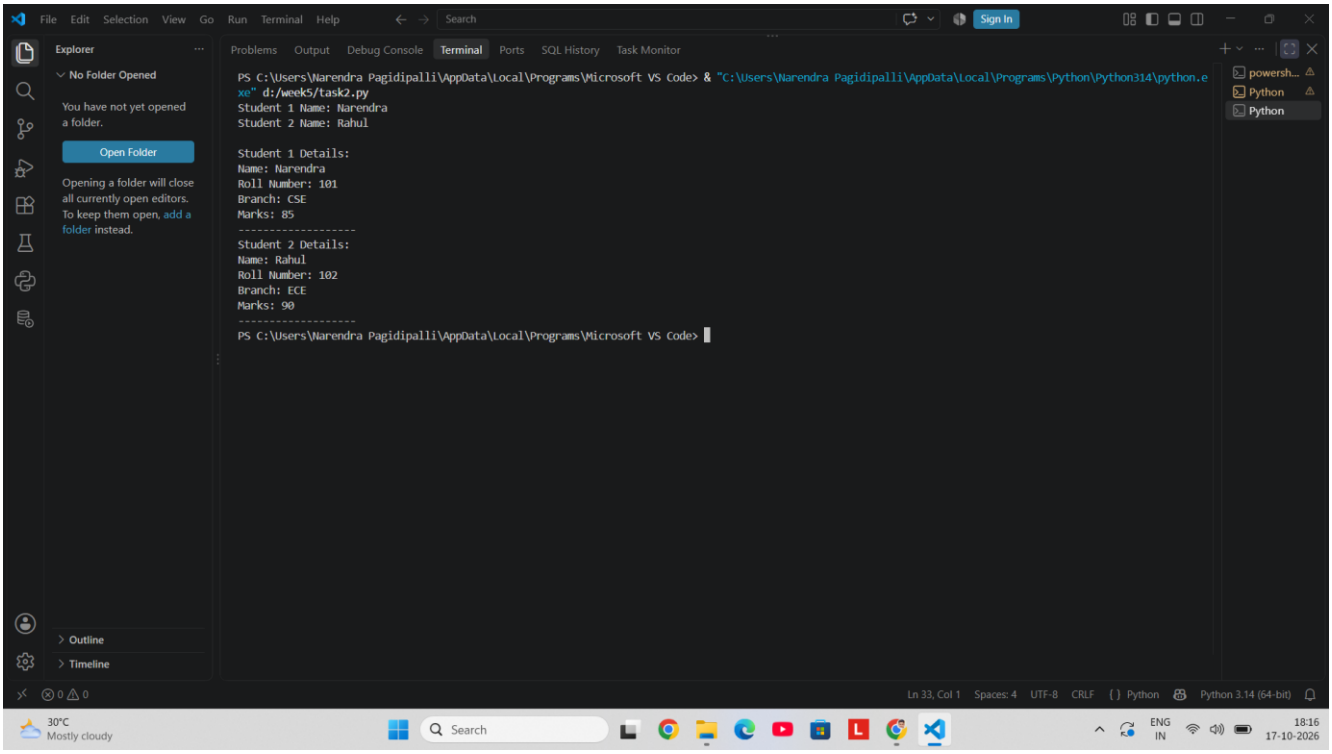

# Creating objects with different values
s1 = Student("Narendra", 101, "CSE", 85)
s2 = Student("Rahul", 102, "ECE", 90)

# Accessing initialized attributes
print("Student 1 Name:", s1.name)
print("Student 2 Name:", s2.name)

# Displaying complete information
print("\nStudent 1 Details:")
s1.display()

print("Student 2 Details:")
s2.display()
```

4. Output



5. Explanation

- Constructor (__init__): It is called automatically when a Student object is created.
- self: It refers to the current object.
- Attributes: name, roll_no, branch, and marks store each student's information.
- Objects: s1 and s2 are created with different values.
- Attribute access: s1.name and s2.name access the initialized names.
- Method: display() prints the complete information of a student.

6. Learning Outcome

I learned how to use the __init__ constructor to initialize attributes, create objects with different values, access attributes, and call methods.

7. Conclusion

This program demonstrates how constructors initialize object data and how objects access and display their information in Python.